

Nakijk 02_01 — V4

NAKIJK V4 · Hoofdstuk 02 · Model, lijst en statuscodes · bij Oefening_02_01.md · vierde versie van je code, 10 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Controllers/ArtikelController.cs` (gewijzigd op 10 oktober om 00:11). `Models/Artikel.cs`, `Program.cs`, `GlobalUsings.cs` en het projectbestand zijn niet gewijzigd sinds V1.

OPMERKING

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in.

Eén uitzondering, op jouw vraag: de routes van de endpoints kreeg je na V3 van mij. Dat stond niet in een nakijkverslag, maar in de chat.

Ik kon je API ook nu niet starten: op de werkruimte bij je laptop staat geen .NET. Ik heb je code gelezen en elk verzoek uit de opgave op papier nagelopen.

In één oogopslag

Punt 1 tot en met 6 zijn in orde. Alle verzoeken uit de opgave voor die punten geven nu het gevraagde antwoord: de juiste statuscode, de juiste artikelen en letterlijk het juiste bericht. Enkel punt 7, de artikelfiche, staat er nog niet in.

Daarnaast blijven er een paar dingen over die geen invloed hebben op het antwoord, maar die je beter opkuißt: twee returntypes (B1), een overbodige controle met een waarschuwing (C1), en de punten uit C2.

Punt	Wat de opgave vraagt	V3	V4
1	het model <code>Artikel</code>	✓ in orde	✓ in orde
2	de controller op <code>api/artikel</code> met minstens zes artikelen	✓ in orde	✓ in orde
3	<code>GET api/artikel</code>	✓ in orde	✓ in orde
4	<code>GET api/artikel/{id}</code>	✗ verkeerd adres	✓ in orde
5	<code>GET api/artikel/soort/{soort}</code>	✗ verkeerd adres	✓ in orde
6	<code>GET api/artikel/voorraad</code>	✗ verkeerd adres	✓ in orde
7	<code>ArtikelFiche</code> en <code>GET api/artikel/fiche/{id}</code>	✗ ontbreekt	✗ ontbreekt (A4)

Fout uit V3	Status in V4
A1, punt 4, 5 en 6: routes beginnen met <code>/</code>	opgelost
A4, punt 7 ontbreekt	nog open
B1, twee methodes beloven één artikel	nog open
C1, overbodige controle in <code>GetVoorraad</code>	nog open
C2, waarschuwing CS8600, naamgeving, <code>.http</code> , lege regels	nog open

Waarom punt 4, 5 en 6 nu werken

Alle drie de routes beginnen nu zonder `/`. ASP.NET zet ze daardoor achter de route van de controller:

Methode	Jouw route	Volledig adres	De opgave vraagt
GetArtikelen	geen	<code>api/Artikel</code>	<code>api/artikel</code> ✓
GetArtikelId	<code>"{id}"</code>	<code>api/Artikel/{id}</code>	<code>api/artikel/{id}</code> ✓
GetArtikelSoort	<code>"soort/{soort}"</code>	<code>api/Artikel/soort/{soort}</code>	<code>api/artikel/soort/{soort}</code> ✓
GetVoorraad	<code>"voorradijg"</code>	<code>api/Artikel/voorradijg</code>	<code>api/artikel/voorradijg</code> ✓

Zo antwoordt je API nu op de verzoeken uit de opgave:

Verzoek	Je API antwoordt nu	Klopt?
GET <code>api/artikel</code>	200 OK met de zes artikelen	✓
GET <code>api/artikel/1</code>	200 OK met artikel 1	✓
GET <code>api/artikel/99</code>	404 Not Found met <i>We vonden geen artikel met Id 99.</i>	✓
GET <code>api/artikel/soort/Elektronica</code>	200 OK met artikel 1 en 2	✓
GET <code>api/artikel/soort/processor</code>	404 Not Found met <i>We vonden geen artikelen van de soort processor.</i>	✓
GET <code>api/artikel/voorradijg</code>	200 OK met artikel 1, 2, 3 en 5	✓
idem, als alle voorraden op 0 staan	200 OK met een lege lijst	✓

Twee dingen die je nu misschien opvallen, en waarom ze kloppen:

- `voorradijg` en `{id}` **zitten elkaar niet in de weg**. Ze hebben allebei drie stukken (*api*, *artikel*, en dan *voorradijg* of een *Id*). ASP.NET geeft dan voorrang aan het vaste woord boven een `{...}`. Een verzoek naar `api/artikel/voorradijg` komt dus bij `GetVoorraad` terecht, niet bij `GetArtikelId`.
- `api/artikel/abc` **geeft 400 Bad Request**, met een JSON-object met de fout. *abc* past in de route `{id}`, maar niet in de parameter `int id`. Omdat je controller `[ApiController]` heeft, kijkt ASP.NET dat na vóór je methode start, en antwoordt het zelf met 400. Je eigen 404-bericht komt dan niet aan bod. De opgave vraagt hier niets over, dus dat is in orde. Je leerboek, hoofdstuk 25, toont hoe een beperking op de route daar een 404 van maakt.
- **De oude adressen bestaan niet meer**. De adressen uit V3, zoals `/1` en `/hallo`, passen nu bij geen enkel endpoint. ASP.NET geeft dan zelf een lege 404, zonder jouw bericht. De 400 bij `/hallo` uit de vragen van V3 is daarmee ook weg.

A. Wat een verkeerd antwoord geeft

A4. Punt 7: de artikelfiche ontbreekt nog

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/fiche/1</code>	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met de fiche van artikel 1
GET <code>api/artikel/fiche/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>

Ook de klasse `ArtikelFiche` staat nog niet in de map `Models`.

Richting, stap voor stap.

- 1. De klasse.** Maak in de map `Models` een eigen bestand voor `ArtikelFiche`, net als `Artikel.cs`. De vier gegevens staan in de tabel van punt 7 van de opgave.
 - *Ja of nee* is in C# een `bool`.
 - *Het artikel zelf* betekent dat dat gegeven van het type `Artikel` is: je eigen klasse.
 - *Geheel getal* is een `int`, zoals bij `AantalOpVoorraad`.
- 2. Het endpoint.** De route volgt dezelfde regel als je andere endpoints; je kreeg ze al in de chat. Het returntype is een fiche, geen artikel.
- 3. De drie vragen die het endpoint beantwoordt.** Twee daarvan los je al op in je andere methodes:
 - *Bestaat het artikel?* Dat doet `GetArtikelId`. Bestaat het niet, dan geef je hetzelfde bericht.
 - *Hoeveel andere artikelen hebben dezelfde soort?* Dat lijkt op de lus in `GetArtikelSoort`.
 - *Is er een ander artikel van die soort dat goedkoper is?* Ook een lus over je lijst, met een andere voorwaarde.
- 4. De fiche samenstellen.** Maak één nieuw `ArtikelFiche`-object, vul de vier gegevens in, en geef het terug met 200.

Drie valkuilen.

- **Het artikel zelf telt niet mee.** Zoek je alle artikelen van dezelfde soort, dan zit het artikel zelf daar ook tussen. Hoe herken je in je lus dat een artikel het artikel van de fiche zelf is? Denk aan wat elk artikel uniek maakt.
- **"Geen enkel ander artikel is goedkoper"** betekent niet "dit is het enige goedkoopste". Hebben twee artikelen dezelfde laagste prijs, dan is bij allebei geen ander artikel goedkoper, dus krijgen ze allebei ja. Het is makkelijker om te zoeken of er een goedkoper artikel bestaat, dan om te bewijzen dat dit het goedkoopste is.
- `IsVoorradig` volgt dezelfde regel als je voorwaarde in `GetVoorraad`: een voorraad groter dan 0. Zet die regel niet anders, anders spreken twee endpoints elkaar tegen.

Zo test je het met jouw artikelen, volgens de drie controles onderaan punt 7:

Controle uit de opgave	Neem dit artikel	Wat je moet zien
een artikel dat als enige van zijn soort in de lijst zit	3, 4, 5 of 6	AantalVanDezelfdeSoort is 0
het artikel waarvan je de voorraad op 0 zette	4 of 6	IsVoorradig is nee
twee artikelen van dezelfde soort	1 en 2 (Elektronica)	IsGoedkoopsteVanDeSoort staat maar bij één van de twee op ja

Twee dingen om op te letten bij het lezen van het antwoord:

- **Ja en nee zie je in JSON als *true* en *false***. Een `bool` wordt zo omgezet. *Nee* in de opgave is dus *false* in `Scalar`.
- **Ook artikel 3 tot 6 krijgen ja bij `IsGoedkoopsteVanDeSoort`**. Is er geen ander artikel van die soort, dan is er ook geen goedkoper. Krijg je daar nee, dan zoekt je code naar iets anders dan de opgave vraagt.

B. Werkt, maar wijkt af

B1. Twee methodes beloven één artikel, maar geven een lijst

Waar: de kop van `GetArtikelSoort` (regel 40) en van `GetVoorraad` (regel 64).

Wat de client krijgt, klopt: een JSON-lijst. Maar het returntype tussen de punthaken zegt dat er **één** `Artikel` terugkomt. C# laat dat toe omdat `Ok(...)` elke waarde aanvaardt. `Scalar` leest het type wel, en toont daardoor één artikel als voorbeeld van het antwoord, terwijl er een lijst komt.

Richting. `GetArtikelen` op regel 20 geeft ook een lijst terug. Kijk wat daar tussen de punthaken staat.

C. Opkuis, zonder invloed op het antwoord

C1. In `GetVoorraad` staat een controle die nooit iets doet

Waar: regel 76 tot 79.

- **De voorwaarde is altijd onwaar.** `Count` is een `int`, en een `int` is nooit `null`. De compiler waarschuwt: *CS0472: The result of the expression is always 'false' since a value of type 'int' is never equal to 'null'.*
- **De controle is ook niet nodig.** Binnen de `if` en na de `if` staat hetzelfde. Een controle heeft enkel zin als het antwoord daarna verschilt. Bij punt 5 verschilt het (404 of 200), bij punt 6 niet (altijd 200).

Richting. Haal weg wat geen verschil maakt. Je endpoint antwoordt daarna exact hetzelfde, en de waarschuwing verdwijnt.

C2. Wat bleef staan uit V1

Waar	Wat	Richting
regel 29	Waarschuwing CS8600: <i>Converting null literal or possible null value to non-nullable type.</i> <code>FirstOrDefault</code> kan <code>null</code> geven, maar het type van je variabele zegt dat dat niet kan. Je code werkt, want je controleert daarna op <code>null</code> .	Kijk hoe de les die variabele schrijft in de <code>ProductController</code> van je theorieproject, of lees in je leerboek, hoofdstuk 17, wat een vraagteken achter een type betekent.
regel 8, 43 en 66	Het veld heet <code>Artikelen</code> , de lijsten in je methodes heten <code>artikelen</code> : één hoofdletter verschil. Velden beginnen in C# met een kleine letter; de les schrijft <code>laptops</code> .	Geef de lijst in elke methode een naam die zegt wat erin zit.
<code>ProgrammingAdvancedOefeningen.http</code>	Verwijst nog naar <code>/weatherforecast</code> .	Weghalen, of vervangen door de verzoeken uit de opgave.
regel 17-18	Twee lege regels na elkaar.	Eén is genoeg.

Na C1 en de eerste rij van C2 hoort je `ArtikelController` geen enkele waarschuwing meer te geven. Je ziet ze in Visual Studio via *View* → *Error List*, met *Warnings* aangezet.

D. Wat beter is dan in V3

- **Alle routes staan juist.** Punt 4, 5 en 6 antwoorden nu op het adres uit de opgave.
- **Je logica stond al goed in V3**, en blijft goed: een 404 voor een onbekend `Id` en een onbekende soort, en altijd 200 voor wat op voorraad is.
- **Je paste alleen aan wat nodig was.** De rest van je code werkt nog zoals in V3. Dat is een goede gewoonte: één fout tegelijk oplossen, zodat je weet welke wijziging wat deed.

Wat je zelf moet kunnen uitleggen. De routes kreeg je van mij, als eenmalige uitzondering. Op het examen krijg je ze niet. Controleer daarom of je deze drie vragen zonder hulp kunt beantwoorden:

1. Welk deel van `api/artikel/soort/Elektronica` komt van de controller, en welk deel van de methode?
2. Wat gebeurt er met de route van de controller als de route van een methode met `/` begint?
3. Met een `/` vooraan kun je het volledige adres ook in elke methode zelf schrijven. Waarom doe je dat beter niet? Denk aan wat er gebeurt als de controller ooit een andere naam of route krijgt.

Lukt een van de drie niet, lees dan het IMPORTANT-kader in V3 en hoofdstuk 25 van je leerboek, *Attributen en routes*. Daar staat de regel met een volledige tabel van adressen.

E. Zelf nakijken

Doe na het oplossen van punt 7 en de opkuis deze verzoeken. Een antwoord klopt pas als het letterlijk overeenkomt met de opgave: statuscode én bericht.

Verzoek	Wat je moet zien	Hoort bij
Scalar openen	onder <i>Artikel</i> vijf endpoints, allemaal beginnend met <i>api/Artikel</i> ; bij soort en voorradig toont het voorbeeld van het antwoord een lijst	B1, A4
GET <code>api/artikel/fiche/3</code>	200, <code>AantalVanDezelfdeSoort 0</code> en <code>IsGoedkoopsteVanDeSoort ja</code>	A4
GET <code>api/artikel/fiche/4</code>	200, <code>IsVoorradig nee (false)</code>	A4
GET <code>api/artikel/fiche/1</code> en <code>/fiche/2</code>	bij precies één van de twee <code>IsGoedkoopsteVanDeSoort ja</code>	A4
GET <code>api/artikel/fiche/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>	A4
alle verzoeken uit <i>Waarom punt 4, 5 en 6 nu werken</i>	nog altijd hetzelfde antwoord	controle dat niets brak
Error List na een build	geen waarschuwing meer in <code>ArtikelController</code>	C1, C2

Met het nakijksript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_01 -Project  
"... \ProgrammingAdvancedOefeningen \ProgrammingAdvancedOefeningen"
```

Het script start je API zelf en stuurt de verzoeken uit de opgave. Het verslag komt in de map `Nakijken` naast je project.

Vragen die je je misschien stelt

Waarom geeft `api/artikel/soort/elektronica` een 404, met een kleine e? Twee verschillende dingen vergelijken hier tekst. ASP.NET kijkt bij het **adres** niet naar hoofdletters: *artikel* en *Artikel* zijn hetzelfde. Maar **jouw voorwaarde** in `GetArtikelSoort` vergelijkt twee teksten met `==`, en dat let wel op hoofdletters. *elektronica* is voor C# dus een andere soort dan *Elektronica*. De opgave vraagt hier niets over, dus het is geen fout. Wil je het toch oplossen: je deed het al in hoofdstuk 01, bij de twee sockets van de compatibiliteitsbalie. In je leerboek staat het in hoofdstuk 1, *Tekst vergelijken*.

Soms zie ik het 404-bericht tussen aanhalingstekens, soms niet. Waarom? Geef je een tekst mee aan `NotFound`, dan kiest ASP.NET hoe het die doorstuurt, volgens wat de client vraagt. Vraagt de client uitdrukkelijk JSON, dan komt de tekst tussen aanhalingstekens, want zo ziet tekst eruit in JSON. Anders komt ze als gewone tekst. Het bericht zelf is in beide gevallen hetzelfde, en dat is wat de opgave vraagt.

Waarom krijg ik een andere volgorde van gegevens in de fiche dan in de tabel van de opgave? De volgorde in JSON volgt de volgorde van de eigenschappen in je klasse. Voor de opgave maakt de volgorde niet uit, maar dezelfde volgorde als in de tabel leest wel makkelijker.

Waarom staan de namen in de JSON met een kleine letter, terwijl mijn eigenschappen met een hoofdletter beginnen? ASP.NET zet de namen bij het omzetten naar JSON om naar een kleine beginletter: `AantalVanDezelfdeSoort` wordt `aantalVanDezelfdeSoort`. Dat heet camelCase en is de standaard in ASP.NET. Je hoeft daar niets voor te doen.

Moet de fiche-methode de lijst opnieuw doorzoeken, of kan ik `GetArtikelId` oproepen? Een endpoint roept je niet op vanuit een ander endpoint: het geeft een `ActionResult` terug, geen artikel. Zoek in de fiche-methode zelf opnieuw, zoals in `GetArtikelId`. Dat je daardoor een stuk dubbel schrijft, is in deze oefening normaal. In hoofdstuk 03 verhuist dat zoekwerk naar een repository, en dan valt het dubbele weg.

Buiten deze oefening

Niet nagekeken, maar opgevallen: in de `OnderdelenController` heet de routeparameter van het codeendpoint anders dan de parameter van de methode. Dat is hetzelfde mechanisme als fout 1 tot 3 in 01_03: de naam tussen de accolades koppelt alleen aan een parameter met precies dezelfde naam.